

AI QUANT-CODER

Intelligent Stock Predictor & Code Assistant

TEAM CREDENTIALS:

Agha Essa Khan — Team Leader & AI Architect

Misbah Ramzan — Quantitative & Python Developer

Submission: AMD Developer Hackathon (Act II)

01. The Challenge & Core Vision

Bridging the gap between financial forecasts and execution models

The Problem

Financial predictions (like charts or forecasts) are often disconnected from script execution.

Developing customized, production-grade strategy bots requires high coding expertise and time.

Accessing powerful large language models (LLMs) locally typically involves paywalls or quota constraints.

The AI Quant-Coder Vision

Create an end-to-end framework integrating predictive models with an automated coder assistant.

Fully exploit local open-source LLMs running on AMD GPU hardware.

Provide zero-quota, secure, and customizable coding pipelines directly to quantitative developers.

Seamless Integration

Binds stock predictions directly with RAG-based code templates.

Open Source Focused

Uses local LLMs (like Llama-3/Mistral) keeping code secure.

02. Core System Architecture

Modular pipelines coordinating predictions, data RAG, and LLM orchestration

1. Data Loading & Technical Analytics

Fetches historical prices from Yahoo Finance API dynamically.

Computes indicator arrays: SMA Crossovers, EMA bands, and RSI.

2. Predictive Forecast Engine

Trains a local statistical model on historical prices to generate future projections.

3. TF-IDF Vector RAG Database

Chunks local open-source trading scripts into a NumPy-based search index.

4. Stream-Enabled LLM Client

Formulates the RAG context and submits requests to local or server API endpoints.

Predictive Pipeline

yfinance -> technical indicators -> ARIMA training.

Generative Pipeline

Vector Search -> Prompt Merger -> Stream Generation.

03. Module Deep Dive: Market Predictor

Empirical forecasting engine using time-series modeling

Time-Series Analytical Engine

Automatically fetches up to 10 years of historical price records for structural training.

Fits an ARIMA(1,1,0) statistical model to handle non-stationary data.

Generates a 30-day forecast with confidence intervals to estimate asset directions.

Technical Signal Tracker

Tracks Simple Moving Averages (20 and 50 days) for crossover bullish/bearish signals.

Computes Relative Strength Index (RSI-14D) to flag overbought/oversold indicators.

Plotly Visualization

Interactive dark-mode chart showing historic closing rates, moving averages, and predictions.

ARIMA Modeling

Robust statistical extrapolation based on Autoregressive structures.

Signal Crossover

Evaluates SMA metrics to confirm momentum trends.

04. Module Deep Dive: RAG Assistant

Combining vector retrieval with real-time financial stats

TF-IDF Search Index

Parses a local database of open-source Python trading bots and scripts.

Segments files into logical blocks based on python function/class boundaries.

Performs cosine similarity matching against natural queries to find reference code.

Predictive Context Merger

Retrieves the active predictor state (ticker close, RSI, forecast target, model type).

Packages these metrics into the prompt as system-level developer instructions.

Augmented Prompt Generator

Generates standard code patterns to feed the LLM client, maximizing structural correctness.

Vector Matching

Retrieves template code with exact functional relevance.

Prompt Augmentation

Enriches LLM query with real-time predictions context.

05. Hackathon Core Innovations

Premium user experience and layout control integrations

Thread-Isolated Background Workers

Spawns distinct generator threads to execute LLM connections in the background.

Streamlit UI remains responsive during generation. User can switch pages freely.

Hot-Swappable Workspace Layouts

Adjust split focus and swap panel positioning (Graph Left / Chat Right vs Chat Left / Graph Right) on-page dynamically.

Active Chat Visual Styling

Active chat sessions stand out with a premium green active gradient and  indicator.

Safe State Initialization

Fresh browser reloads automatically spawn clean empty chats while keeping history.

Zero Lock UI

Multi-session threads prevent the Streamlit screen from blocking.

Dynamic Workspace

Align panels and focus sizes directly inside the app.

06. AMD GPU Hardware & ROCm Optimizations

Transitioning local prototypes to high-throughput cloud scale

Ollama Local Setup (Prototyping)

Local desktop setup using CPU/APU execution with Ollama.

AMD Developer Cloud (GPU Production Scale)

Transition the workspace to an AMD Developer Cloud Ubuntu server.

Powered by high-end AMD CDNA GPUs (MI210, MI250, MI300).

vLLM & ROCm 6.x Setup

Serve heavy open-source models (like Llama-3-70B) with high throughput.

vLLM utilizes PagedAttention and ROCm 6.x visibility for maximum token speeds.

Streamlit interface seamlessly targets the server via OpenAI API-compatible endpoint.

ROCm Visibility

Direct hardware interface for fast model inference.

vLLM Serving

Serves institutional models with tenant security and speed.

07. Future Roadmap & Scaling

Next phases of the AI Quant-Coder platform

1. Sandbox Python Execution Engine

Execute the generated Python code securely inside an isolated Docker sandbox.

Plot backtested strategy returns directly inside the Streamlit interface.

2. Multi-Model Time-Series Forecasting

Integrate LSTM (Deep Learning) and Prophet alongside ARIMA models.

Allow developers to select their forecast mathematical framework.

3. Real-Time Exchange Integrations

Allow paper trading and direct script deployment to stock and crypto brokers.

4. Advanced Vector Databases

Integrate ChromaDB/FAISS to support massive code script datasets.

Strategy Backtester

Run and plot code metrics inside the app sandbox.

LSTM / Deep Learning

Expand time-series capabilities to neural networks.

THANK YOU

AI Quant-Coder: Empowering traders with predictive automated coding.

Developed by Agha Essa Khan & Misbah Ramzan